



Red Hat OpenShift AI Self-Managed 3.4

OpenShift AI tutorial - Fraud detection example

Use OpenShift AI to train and deploy an example fraud detection model

Red Hat OpenShift AI Self-Managed 3.4 OpenShift AI tutorial - Fraud detection example

Use OpenShift AI to train and deploy an example fraud detection model

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Step-by-step guidance for using OpenShift AI to train an example model in JupyterLab, deploy the model, refine the model by using automated pipelines, and train the model by using distributed computing frameworks.

Table of Contents

CHAPTER 1. INTRODUCTION	3
1.1. ABOUT THE EXAMPLE FRAUD DETECTION MODEL	3
1.2. BEFORE YOU BEGIN	3
CHAPTER 2. SETTING UP A PROJECT AND STORAGE	4
2.1. NAVIGATING TO THE OPENSIFT AI DASHBOARD	4
2.2. SETTING UP YOUR PROJECT	5
2.3. STORING DATA WITH CONNECTIONS	6
2.3.1. Running a script to install local object storage buckets and create connections	7
2.3.2. (Optional) Creating connections to your own S3-compatible object storage	11
2.4. (OPTIONAL) ENABLING AI PIPELINES	15
2.5. (OPTIONAL) SETTING UP KUEUE RESOURCES	18
CHAPTER 3. CREATING A WORKBENCH AND USING NOTEBOOKS	21
3.1. CREATING A WORKBENCH	21
3.2. IMPORTING THE TUTORIAL FILES INTO THE JUPYTERLAB ENVIRONMENT	24
3.3. RUNNING CODE IN A NOTEBOOK	27
3.4. TRAINING A MODEL	29
CHAPTER 4. DEPLOYING AND TESTING A MODEL	30
4.1. PREPARING THE MODEL FOR DEPLOYMENT	30
4.2. DEPLOYING THE MODEL	30
4.3. TESTING THE MODEL API	32
CHAPTER 5. IMPLEMENTING PIPELINES	34
5.1. AUTOMATING WORKFLOWS WITH AI PIPELINES	34
5.1.1. Creating a pipeline	34
5.1.2. Adding nodes to your pipeline	35
5.1.3. Specifying the training file as a dependency	36
5.1.4. Creating and storing the ONNX-formatted output file	38
5.1.5. Configuring the connection to storage	38
5.1.6. Running your pipeline	42
5.2. RUNNING A PIPELINE GENERATED FROM PYTHON CODE	45
CHAPTER 6. RUNNING A DISTRIBUTED WORKLOAD	49
6.1. DISTRIBUTING TRAINING JOBS	49
6.1.1. Distributing training jobs with Ray	49
6.1.2. Distributing training jobs with the Training Operator	50
CHAPTER 7. CONCLUSION	52

CHAPTER 1. INTRODUCTION

Welcome! In this tutorial, you learn how to use data science, artificial intelligence (AI), and machine learning (ML) in an OpenShift development workflow.

You complete the following tasks in [Red Hat OpenShift AI](#) without installing any software on your computer:

- Explore a pre-trained fraud detection model by using a Jupyter notebook.
- Deploy the model by using OpenShift AI model serving.
- Refine and train the model by using automated pipelines.
- Learn how to train the model by using distributed computing frameworks.

1.1. ABOUT THE EXAMPLE FRAUD DETECTION MODEL

The example fraud detection model monitors credit card transactions for potential fraudulent activity. It analyzes the following credit card transaction details:

- The geographical distance from an earlier credit card transaction.
- The price of the current transaction, compared to the median price of all the transactions.
- Whether the user completed the transaction by using the hardware chip in the credit card, by entering a PIN number, or by making an online purchase.

Based on this data, the model outputs the likelihood of the transaction being fraudulent.

1.2. BEFORE YOU BEGIN

You must have access to an OpenShift cluster which has Red Hat OpenShift AI installed.



NOTE

Due to resource constraints, this v3.2 edition of the tutorial is not supported in the [Red Hat Developer Sandbox](#).



IMPORTANT

If your cluster uses self-signed certificates, before you begin the tutorial, your OpenShift AI administrator must add self-signed certificates for OpenShift AI as described in [Working with certificates](#) (Self-Managed).

If you're ready, [start the tutorial](#).

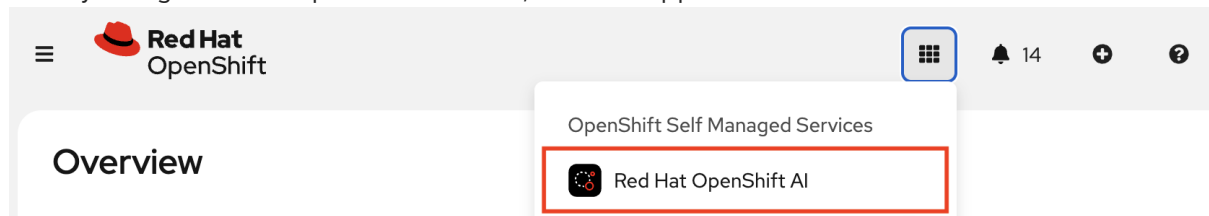
CHAPTER 2. SETTING UP A PROJECT AND STORAGE

2.1. NAVIGATING TO THE OPENSIFT AI DASHBOARD

You can access the OpenShift AI dashboard from the OpenShift console. From the dashboard, you can verify that the installed OpenShift AI version matches the version used in this tutorial.

Procedure

1. After you log in to the OpenShift console, click the application launcher icon on the header.



2. Log in to the OpenShift AI dashboard by using your OpenShift credentials. OpenShift AI uses the same credentials as OpenShift for the dashboard, notebooks, and all other components. The OpenShift AI dashboard shows the **Home** page.



NOTE

You can navigate back to the OpenShift console by clicking the application launcher.

Red Hat Applications

 OpenShift Console

 OpenShift Cluster Manager

For now, stay in the OpenShift AI dashboard.

3. Make sure that the version of this tutorial matches the OpenShift AI version on your cluster. Red Hat OpenShift AI v3.2 is the version described in this tutorial.

To find the OpenShift AI version for your cluster:

- a. In the top navigation bar of the OpenShift AI dashboard, next to your username, click the help icon () and then select **About**. The **About** page shows the installed version.
4. Check the [Red Hat OpenShift AI Supported Configurations for 3.x](#) page to verify that your installed version is a supported version.
 5. If this version of the tutorial does not match the installed version, access the matching version of the tutorial:
 - a. Navigate to the [Red Hat OpenShift AI documentation page](#).
 - b. Select the matching version from the drop-down list.

Next step

[Setting up your project](#)

2.2. SETTING UP YOUR PROJECT

To implement a data science workflow, you must create a project as described in the following procedure. Projects help your team to organize and work together on resources within separated namespaces. From a project, you can create many workbenches, each with its own IDE environment (for example, JupyterLab), and each with its own connections and cluster storage. In addition, the workbenches can share models and data with pipelines and model servers.

Prerequisites

- You have logged in to **Red Hat OpenShift AI**.

Procedure

1. From the left navigation menu, select **Projects**. This page lists any existing projects that you have access to. You can select an existing project (if any) or create a new one.
2. Click **Create project**.



NOTE

You can start a Jupyter notebook by clicking the **Start basic workbench** button. However, in that case, it is a one-off Jupyter notebook run in isolation.

3. In the **Create project** modal, enter a display name and description.

4. Click **Create**.

Verification

Your project opens in the dashboard.

You can click the tabs to view more information about the project components and project access permissions:

+ image::projects/ds-project-new.png[New project]

- **Workbenches** are instances of your development and experimentation environment. They typically contain individual development environments (IDEs), such as JupyterLab, RStudio, and Code Server.
- **Pipelines** are a structured series of processes that collect, process, analyze, and visualize data. With AI pipelines, you can automate the execution of notebooks and Python code. By using pipelines, you can run long training jobs or retrain your models on a schedule without having to manually run them in a notebook.
- **Deployments** for quickly serving a trained model. A model server is a container image for a machine learning model. It exposes APIs to receive data, run the data through a trained model, and delivers a result (for example, a fraud alert).
- **Cluster storage** is a persistent volume that retains the files and data you're working on within a workbench. A workbench has access to one or more cluster storage instances.
- **Connections** contain object data which you can use for purposes such as configuration parameters and storing models, data, or artifacts.
- **Permissions** define which users and groups can access the project.

Next step

[Storing data with connections](#)

2.3. STORING DATA WITH CONNECTIONS

Add connections to workbenches to connect your project to data inputs and object storage buckets. A connection is a resource that has the configuration parameters needed to connect to a data source or data sink, such as an AWS S3 object storage bucket.

For this tutorial, you run a provided script that creates the following local MinIO storage buckets for you:

- **My Storage** - Use this bucket for storing your models and data. You can reuse this bucket and its connection for your notebooks and model servers.
- **Pipelines Artifacts** - Use this bucket as storage for your pipeline artifacts. When you create a pipeline server, you need a pipeline artifacts bucket. For this tutorial, create this bucket to separate it from the first storage bucket for clarity.

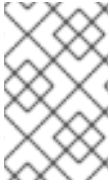


NOTE

Although you can use one storage bucket for both storing models and data and for storing pipeline artifacts, this tutorial follows the recommended practice of using separate storage buckets for each purpose.

The provided script also creates a connection to each storage bucket.

To run the script that installs local MinIO storage buckets and creates connections to them, follow the steps in [Running a script to install local object storage buckets and create connections](#) .

**NOTE**

If you want to use your own S3-compatible object storage buckets (instead of using the provided script), follow the steps in [Creating connections to your own S3-compatible object storage](#).

2.3.1. Running a script to install local object storage buckets and create connections

If you do not have your own S3-compatible storage or if you want to use a disposable local MinIO instance instead, run a script (provided in the following procedure) that automatically completes these tasks:

- Creates a MinIO instance in your project.
- Creates two storage buckets in that MinIO instance.
- Generates a random user id and password for your MinIO instance.
- Creates two connections in your project, one for each bucket and both using the same credentials.
- Installs required network policies for service mesh functionality.

This script is based on the [guide for deploying MinIO](#).

**IMPORTANT**

The MinIO-based Object Storage that the script creates is **not** meant for production usage.

**NOTE**

If you want to connect to your own storage, see [Creating connections to your own S3-compatible object storage](#).

Prerequisites

- You must know the OpenShift resource name for your project so that you run the provided script in the correct project. To get the project's resource name:
In the OpenShift AI dashboard, select **Projects** and then click the ? icon next to the project name. A text box opens with information about the project, including its resource name:

[Fraud Detection](#) ?

My Fraud Detection project

Resource names and types are used to find your resources in OpenShift.

Resource name fraud-detection

Resource type Project



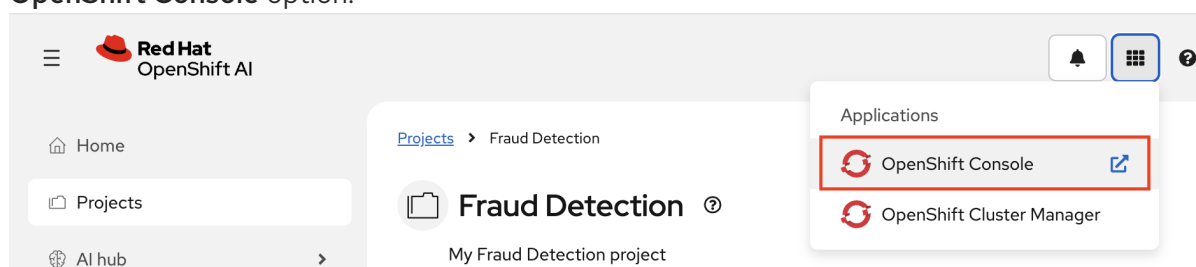
NOTE

The following procedure describes how to run the script from the OpenShift console. If you are knowledgeable in OpenShift and can access the cluster from the command line, instead of following the steps in this procedure, you can use the following command to run the script:

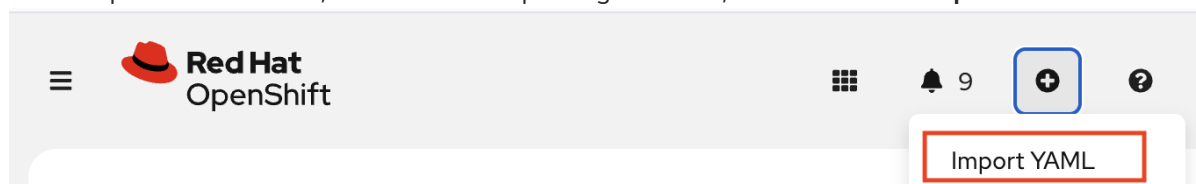
```
oc apply -n <your-project-name> -f https://github.com/rh-aisservices-bu/fraud-detection/raw/v3.2/setup/setup-s3-no-sa.yaml
```

Procedure

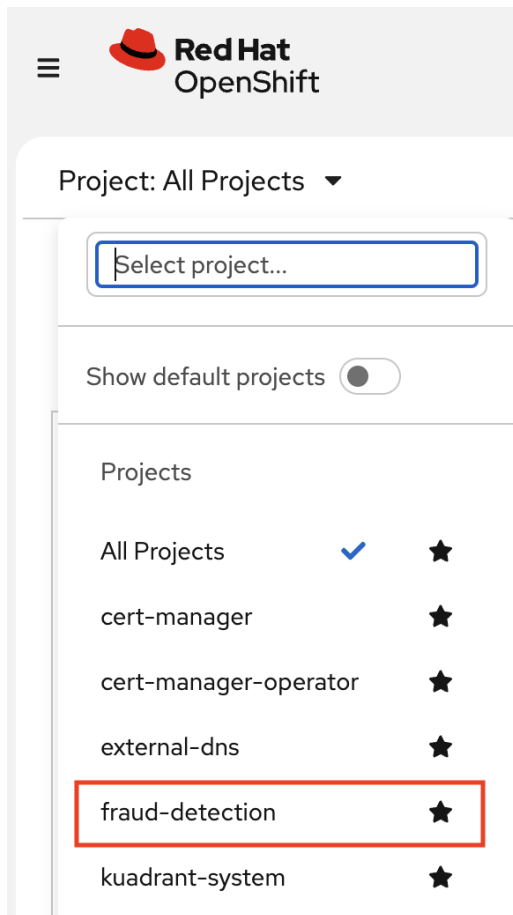
1. In the OpenShift AI dashboard, click the application launcher icon and then select the **OpenShift Console** option.



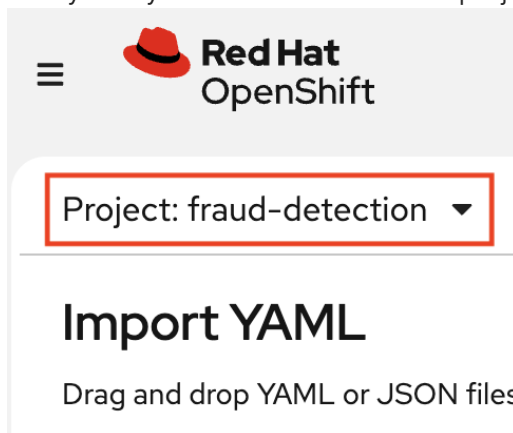
2. In the OpenShift console, click + in the top navigation bar, and then click **Import YAML**.



3. Click the down arrow next to the project name, and then select your project from the list of projects. If needed, type the name of your project in the **Select project** search field.



4. Verify that you selected the correct project.



5. Copy the following code and paste it into the **Import YAML** editor.



NOTE

This code gets and applies the **setup-s3-no-sa.yaml** file.

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: demo-setup
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
```

```

metadata:
  name: demo-setup-edit
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: edit
subjects:
  - kind: ServiceAccount
    name: demo-setup
---
apiVersion: batch/v1
kind: Job
metadata:
  name: create-s3-storage
spec:
  selector: {}
  template:
    spec:
      containers:
        - args:
            - -ec
            - |-
              echo -n 'Setting up MinIO instance and connections'
              oc apply -f https://github.com/rh-aisservices-bu/fraud-detection/raw/v3.2/setup/setup-
s3-no-sa.yaml
          command:
            - /bin/bash
          image: image-registry.openshift-image-registry.svc:5000/openshift/tools:latest
          imagePullPolicy: IfNotPresent
          name: create-s3-storage
        restartPolicy: Never
      serviceAccount: demo-setup
      serviceAccountName: demo-setup

```

6. Click **Create**.

Verification

1. In the OpenShift console, there is a "Resources successfully created" message and a list of the following resources:
 - **demo-setup**
 - **demo-setup-edit**
 - **create-s3-storage**
2. In the OpenShift AI dashboard:
 - a. Select **Projects** and then click the name of your project, **Fraud detection**.
 - b. Click **Connections**. There are two connections listed: **My Storage** and **Pipeline Artifacts**.


Fraud Detection ⓘ

My Fraud Detection project

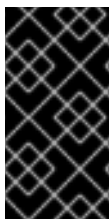
Actions ▾

Overview Workbenches Pipelines Models Cluster storage **Connections** Permissions Settings


Connections ⓘ

Add connection

Name ↑	Type ↑	Model serving compatibility	Connected resources
My Storage ⓘ	S3 compatible object storage - v1	S3 compatible object storage	-
Pipeline Artifacts ⓘ	S3 compatible object storage - v1	S3 compatible object storage	-



IMPORTANT

If your cluster uses self-signed certificates, your OpenShift AI administrator might need to configure a certificate authority (CA) to securely connect to the S3 object storage, as described in [Accessing S3-compatible object storage with self-signed certificates](#) (Self-Managed).

Next steps

- Decide whether you want to complete the pipelines section of this tutorial. With OpenShift AI pipelines, you can automate the execution of your notebooks and Python code. You can run long training jobs or retrain your models on a schedule without having to manually run them in a notebook.
If you want to complete the pipelines section of this tutorial, go to [Enabling AI pipelines](#).
- Decide whether you want to complete the *Distributing training jobs with the Training Operator* section of this tutorial. In that section, you implement a distributed training job by using Kueue for managing job resources.
If you want to complete the *Distributing training jobs with the Training Operator* section of this tutorial, go to [Setting up Kueue resources](#).
- Otherwise, skip to [Creating a workbench](#).

2.3.2. (Optional) Creating connections to your own S3-compatible object storage



NOTE

Skip this procedure if you completed the steps in [Running a script to install local object storage buckets and create connections](#).

If you have existing S3-compatible storage buckets that you want to use for this tutorial, you must create a connection to one storage bucket for saving your data and models. If you want to complete the pipelines section of this tutorial, create another connection to a different storage bucket for saving pipeline artifacts.

Prerequisites

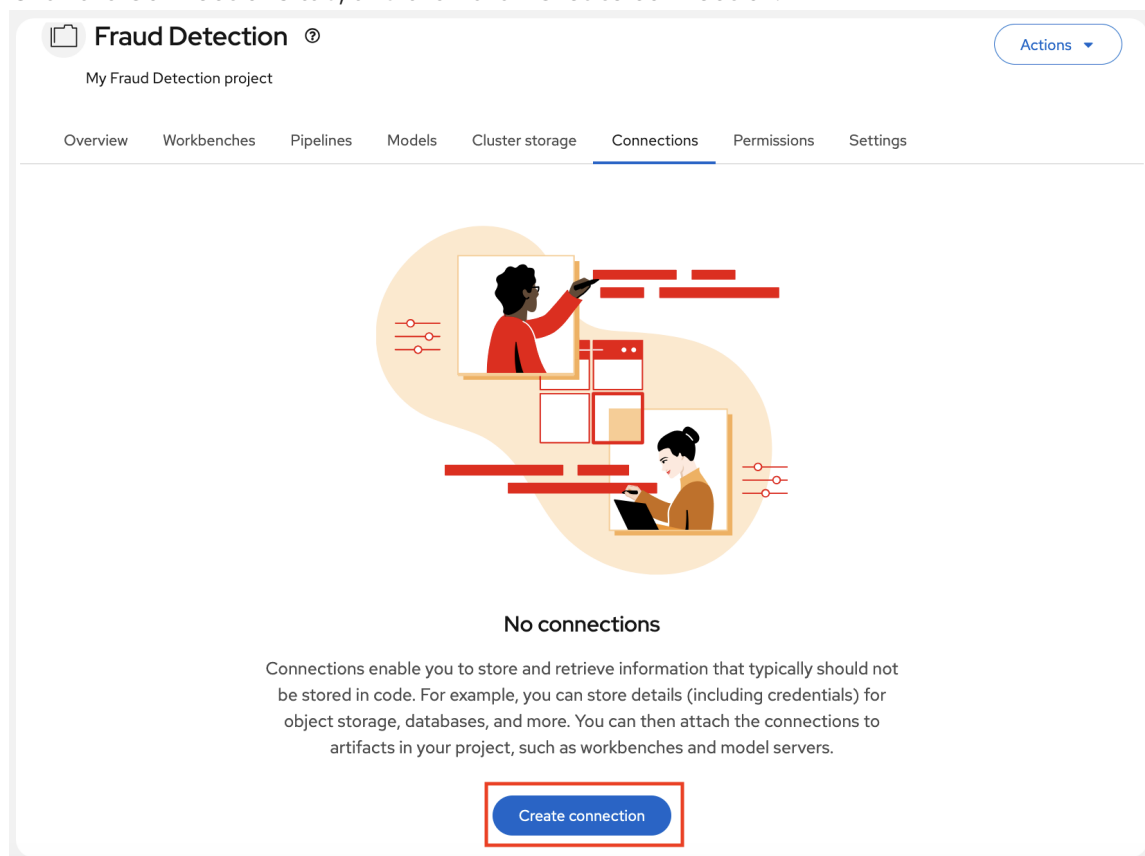
To create connections to your existing S3-compatible storage buckets, you need the following credential information for the storage buckets:

- Endpoint URL
- Access key
- Secret key
- Region
- Bucket name

If you do not have this information, contact your storage administrator.

Procedure

1. Create a connection for saving your data and models:
 - a. In the OpenShift AI dashboard, navigate to the page for your project.
 - b. Click the **Connections** tab, and then click **Create connection**.



- c. In the **Add connection** modal, for the **Connection type** select **S3 compatible object storage - v1**.
- d. Complete the **Add connection** form and name your connection **My Storage**. This connection is for saving your personal work, including data and models.

Add connection ✕

Connection type *

S3 compatible object storage - v1 ▼

Connect to storage systems that are compatible with Amazon S3, enabling integration with other S3-compatible services and applications.
[View connection type details](#)

Connection details

Connection name *

My Storage

The resource name will be **my-storage**.

[Edit resource name](#) ?

Connection description

Access key * ?

kadhaksjdhsuidacn

Secret key * ?

..... 🗑

Endpoint * ?

https://minio-s3-fraud-detection-example.com

Region ?

us-east

Create **Cancel**

e. Click **Create**.

2. Create a connection for saving pipeline artifacts:



NOTE

If you do not intend to complete the pipelines section of the tutorial, you can skip this step.

a. Click **Add connection**.

b. Complete the form and name your connection **Pipeline Artifacts**.

Add connection ✕

Connection type *

S3 compatible object storage - v1 ▼

Connect to storage systems that are compatible with Amazon S3, enabling integration with other S3-compatible services and applications.
[View connection type details](#)

Connection details

Connection name *

Pipeline Artifacts

The resource name will be **pipeline-artifacts**.

[Edit resource name](#) ?

Connection description

Access key * ?

yuieayoaifiasbyr5jkskdhka

Secret key * ?

..... 🗨

Endpoint * ?

https://minio-s3-fraud-detection-example.com

Region ?

us-east

Create **Cancel**

- c. Click **Create**.

Verification

In the **Connections** tab for the project, check to see that your connections are listed.


Fraud Detection ⓘ

My Fraud Detection project

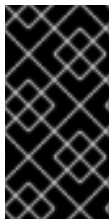
Actions ▾

Overview Workbenches Pipelines Models Cluster storage **Connections** Permissions Settings


Connections ⓘ

Add connection

Name ↑	Type ↑	Model serving compatibility	Connected resources
My Storage ⓘ	S3 compatible object storage - v1	S3 compatible object storage	-
Pipeline Artifacts ⓘ	S3 compatible object storage - v1	S3 compatible object storage	-



IMPORTANT

If your cluster uses self-signed certificates, your OpenShift AI administrator might need to provide a certificate authority (CA) to securely connect to the S3 object storage, as described in [Accessing S3-compatible object storage with self-signed certificates](#) (Self-Managed).

Next steps

- Decide whether you want to complete the pipelines section of this tutorial. With OpenShift AI pipelines, you can automate the execution of your notebooks and Python code. You can run long training jobs or retrain your models on a schedule without having to manually run them in a notebook.
If you want to complete the pipelines section of this tutorial, go to [Enabling AI pipelines](#).
- Decide whether you want to complete the *Distributing training jobs with the Training Operator* section of this tutorial. In that section, you implement a distributed training job by using Kueue for managing job resources.
If you want to complete the *Distributing training jobs with the Training Operator* section of this tutorial, go to [Setting up Kueue resources](#).
- Otherwise, skip to [Creating a workbench](#).

2.4. (OPTIONAL) ENABLING AI PIPELINES

You must prepare your tutorial environment to use ai pipelines.



NOTE

If you do not intend to complete the pipelines section of this tutorial, you can skip to the next section, [Setting up Kueue resources](#).

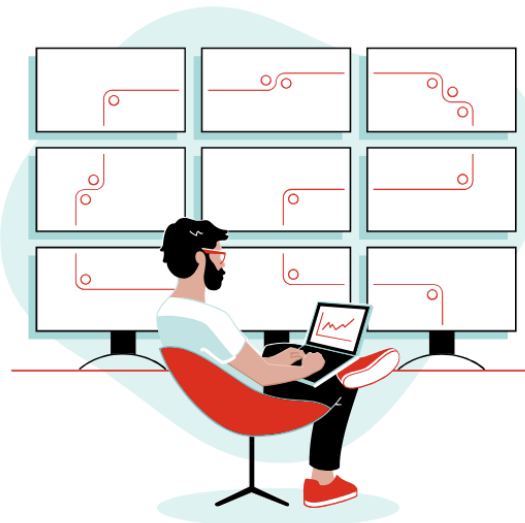
Later in this tutorial, you implement an example pipeline by using the JupyterLab Elyra extension. Elyra enables you to create a visual, end-to-end pipeline workflow that runs in OpenShift AI.

Prerequisites

- You have installed local object storage buckets and created connections, as described in [Storing data with connections](#).

Procedure

1. In the OpenShift AI dashboard, on the **Fraud Detection** page, click the **Pipelines** tab.
2. Click **Configure pipeline server**.



Configure a pipeline server

To create or use pipelines, you must first configure a pipeline server in this project. A pipeline server provides the infrastructure necessary for the pipeline to execute steps, track results, and manage runs.

Configure pipeline server

3. In the **Configure pipeline server** form, click **Autofill from connection** and then click **Pipeline Artifacts**.
The **Configure pipeline server** form fills with credentials for the connection.

4. In the **Advanced Settings** section, leave the default values.
5. Click **Configure pipeline server**.
6. Wait until the loading spinner disappears and **Start by importing a pipeline** is displayed.



IMPORTANT

You must wait until the pipeline configuration is complete before you continue and create your workbench. If you create your workbench before the pipeline server is ready, your workbench cannot submit pipelines to it.

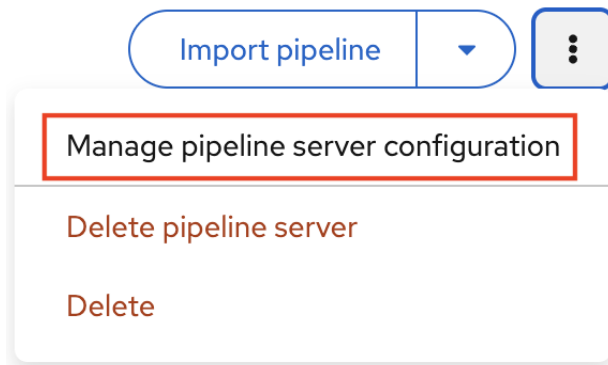
- a. You can click **View progress and event logs**

If you have waited more than 5 minutes, and the pipeline server configuration does not complete, you can click **Cancel pipeline server setup** and create it again.

You can also ask your OpenShift AI administrator to verify that they applied self-signed certificates on your cluster as described in [Working with certificates](#) (Self-Managed).

Verification

1. Navigate to the **Pipelines** tab for the project.
2. Next to **Import pipeline**, click the action menu (:) and then select **Manage pipeline server configuration**.



An information box opens and displays the object storage connection information for the pipeline server.

Next step

If you want to complete the *Distributing training jobs with the Training Operator* section of this tutorial, go to [Setting up Kueue resources](#).

Otherwise, skip to [Creating a workbench](#).

2.5. (OPTIONAL) SETTING UP KUEUE RESOURCES

Distributed training in OpenShift AI uses the Red Hat build of Kueue for admission and scheduling. Before you run the Ray or Training Operator examples in this tutorial, a cluster administrator must install and configure the Red Hat build of Kueue Operator as described in [Kueue workflow](#).

Also, you must prepare your tutorial environment so that you can use Kueue for distributing training with the Training Operator.

In the *Distributing training jobs with the Training Operator* section of this tutorial, you implement a distributed training job by using Kueue for managing job resources. With Kueue, you can manage cluster resource quotas and how different workloads consume them.

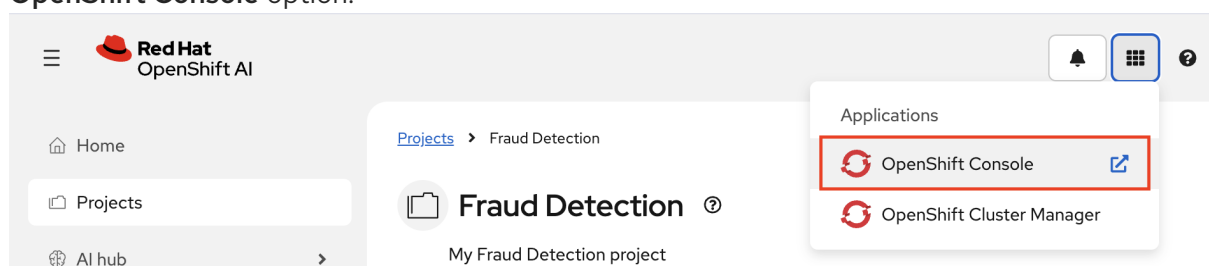


NOTE

If you do not intend to use Kueue to schedule your training jobs in the *Distributing training jobs with the Training Operator* section of this tutorial, skip this procedure and continue to the next section, [Creating a workbench and selecting a workbench image](#).

Procedure

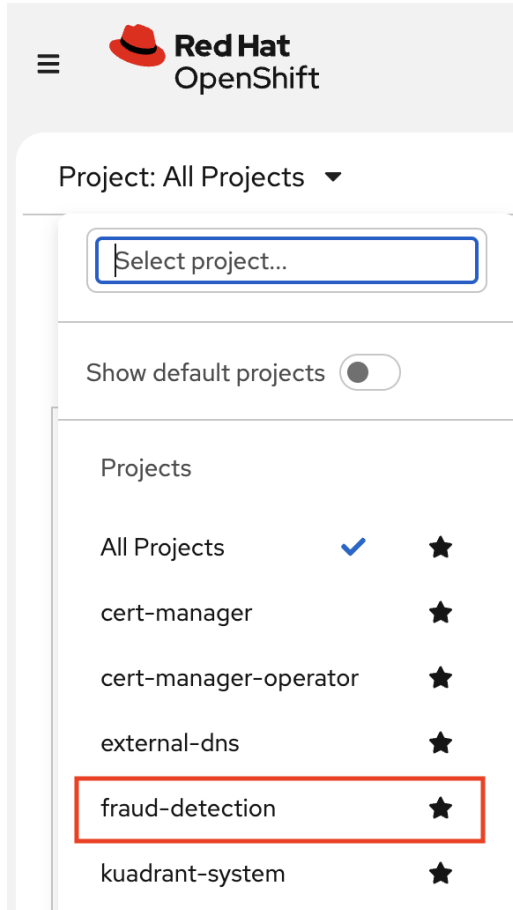
1. In the OpenShift AI dashboard, click the application launcher icon and then select the **OpenShift Console** option.



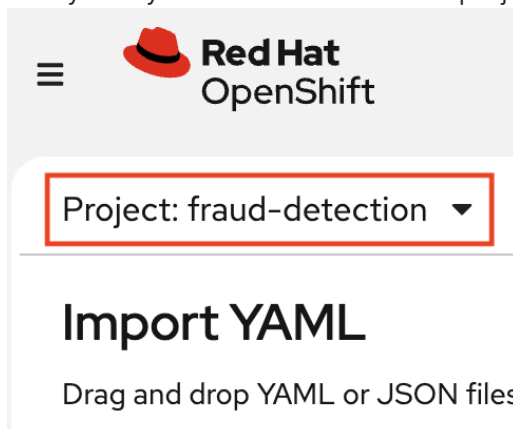
2. In the OpenShift console, click + in the top navigation bar, and then click **Import YAML**.



- Click the down arrow next to the project name, and then select your project from the list of projects. If needed, type the name of your project in the **Select project** search field.



- Verify that you selected the correct project.



- Copy the following code and paste it into the **Import YAML** editor.

```
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: ResourceFlavor
metadata:
  name: default-flavor
```

```
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: ClusterQueue
metadata:
  name: cluster-queue
spec:
  namespaceSelector: {}
  resourceGroups:
  - coveredResources: ["cpu", "memory"]
    flavors:
    - name: "default-flavor"
      resources:
      - name: "cpu"
        nominalQuota: 4
      - name: "memory"
        nominalQuota: 8Gi
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: LocalQueue
metadata:
  name: local-queue
spec:
  clusterQueue: cluster-queue
```

6. Click **Create**.

Verification

- The OpenShift console displays a "Resources successfully created" message with a list of the following resources:
- **default-flavor**
- **cluster-queue**
- **local-queue**

Next step

[Creating a workbench and using notebooks](#)

CHAPTER 3. CREATING A WORKBENCH AND USING NOTEBOOKS

3.1. CREATING A WORKBENCH

A workbench is an instance of your development and experimentation environment. When you create a workbench, you select a workbench image that has the tools and libraries that you need for developing models.

Prerequisites

- You created a **My Storage** connection as described in [Storing data with connections](#).
- If you intend to complete the pipelines section of this tutorial, you configured a pipeline server as described in [Enabling AI pipelines](#).
- If you intend to complete the *Distributing training jobs with the Training Operator* section of this tutorial, you completed the procedure described in [Setting up Kueue resources](#).

Procedure

1. Navigate to the project detail page for the project that you created in [Setting up your project](#).
2. Click the **Workbenches** tab, and then click the **Create workbench** button.



Start by creating a workbench

A workbench is an isolated area where you can work with models in your preferred IDE, such as a Jupyter notebook. You can add accelerators and data connections, create pipelines, and add cluster storage in your workbench.

Create workbench

3. Enter the name and description.

Name *

Fraud Detection

The resource name will be **fraud-detection**.[Edit resource name](#) ?

Description

My Fraud Detection workbench

Red Hat provides several supported workbench images. In the **Workbench image** section, you can select one of the default images or a custom image that an administrator has set up for you. The **Tensorflow** image has the libraries needed for this tutorial.

- Select the latest **Tensorflow** image.

Workbench image

Image selection *

Jupyter | TensorFlow | CUDA | Python 3.12

Version selection *

2025.2

Software: CUDA v12.8, Python v3.12, TensorFlow v2.20

Hover over a version to view its included packages.

? [View package information](#)

- Under the **Deployment size** section, for **Hardware profile**, select **default-profile**.
- Keep the default environment variables and storage options.

Environment variables+ [Add variable](#)**Cluster storage**

Attach existing storage

Create storage

Name

Storage size

Mount path

fraud-detection-storage ⓘ

Max 20GiB

/opt/app-root/src/



- For **Connections**, click **Attach existing connection**.

8. Select **My Storage** (the object storage that you configured earlier) and then click **Attach**.

Connections [Attach existing connections](#) [Create connection](#)

Name ↑	Type ↑	
My Storage ?	S3 compatible object storage - v1	⋮

9. Click **Create workbench**.

Verification

In the **Workbenches** tab for the project, the status of the workbench changes from **Starting** to **Running**.

[Projects](#) > Fraud Detection

Fraud Detection ? [Actions](#) ▾

My Fraud Detection project

Overview **Workbenches** Pipelines Deployments Cluster storage Connections Permissions

Workbenches ? [Create workbench](#)

Name ↑	Workbench image	Hardware profile	Status ↑	
> Fraud Detection ?	Jupyter TensorFlow CUDA Python 3.12 2025.2 ... ✓ Latest	default-profile	Running	Stop ⋮



NOTE

If you made a mistake, you can edit the workbench to make changes.

Status ↑

Running [Stop](#) ⋮

Edit workbench

Delete workbench

Next step

[Importing the tutorial files into the JupyterLab environment](#)

3.2. IMPORTING THE TUTORIAL FILES INTO THE JUPYTERLAB ENVIRONMENT

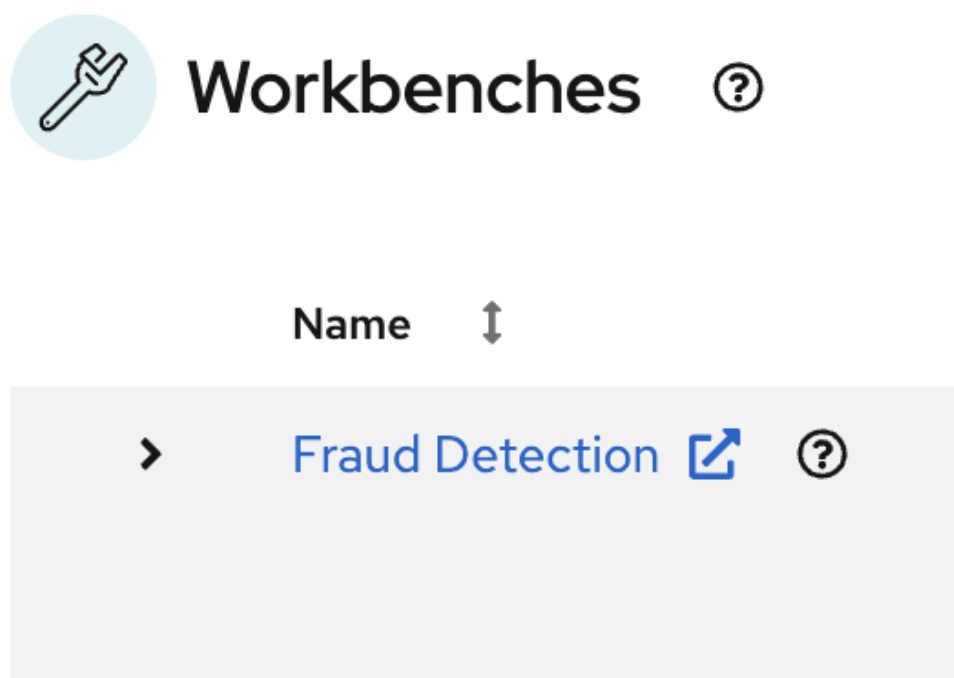
The JupyterLab environment is web-based, but all operations run on Red Hat OpenShift AI and are backed by the OpenShift cluster. This configuration allows you to run notebooks without installing local tools or consuming local CPU, GPU, or memory resources.

Prerequisites

You created a workbench, as described in [Creating a workbench and selecting a workbench image](#).

Procedure

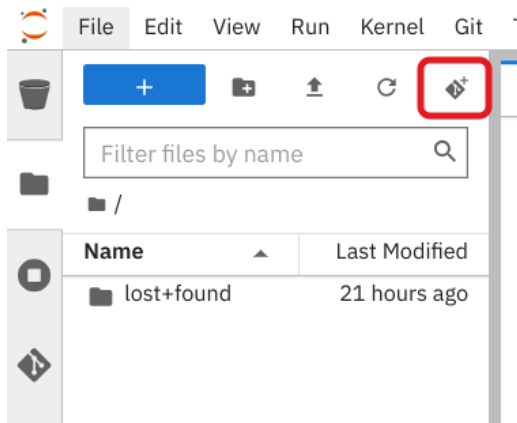
1. In the **Workbenches** tab for your project, click the link for your workbench. If prompted, log in and allow JupyterLab to authorize your user.



Your JupyterLab environment window opens.

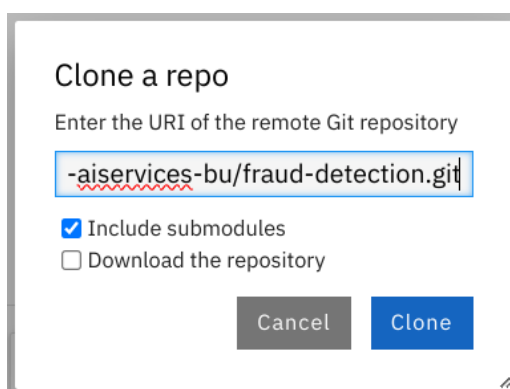
This file-browser window shows the files and folders that are saved inside your own personal space in OpenShift AI.

2. Bring the content of this tutorial inside your JupyterLab environment:
 - a. On the toolbar, click the **Git Clone** icon:

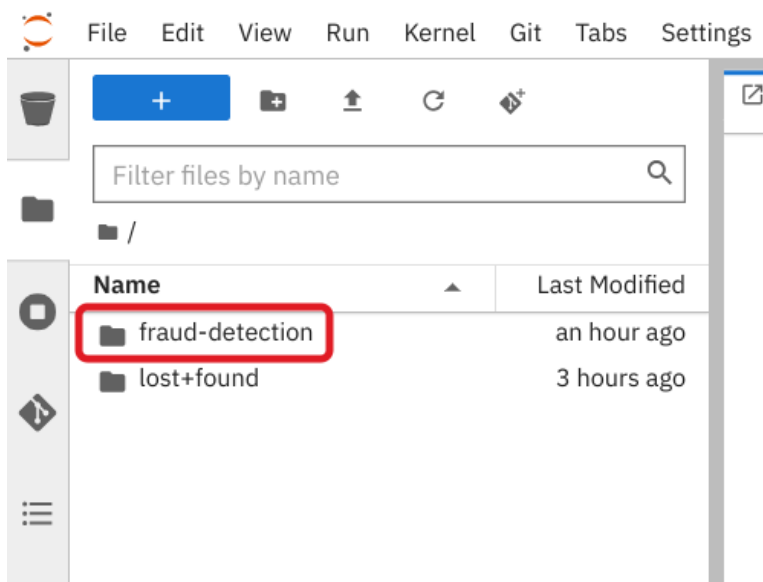


- b. Enter the following tutorial Git **https** URL:

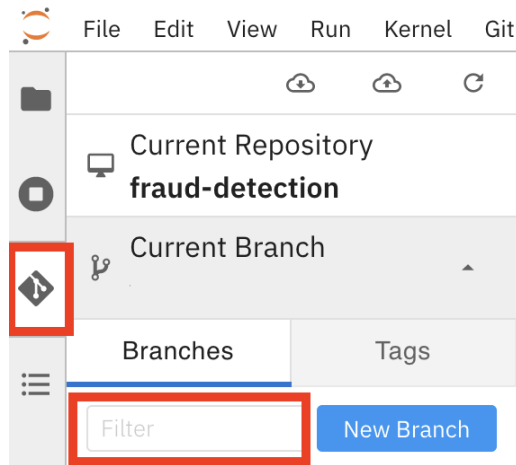
`https://github.com/rh-aishervices-bu/fraud-detection.git`



- c. Select the **Include submodules** option, and then click **Clone**.
- d. In the file browser, double-click the newly-created **fraud-detection** folder to expand its contents.



- e. In the left navigation bar, click the **Git** icon, and then click **Current Branch** to expand the branches and tags selector panel.
- i. On the **Branches** tab, in the **Filter** field, enter **v3.2**.



- f. Select **origin/v3.2**.

The current branch changes to **v3.2**.

Verification

1. In the left navigation bar, click the file browser icon to view the notebooks that you cloned from Git.

■ / fraud-detection /

Name

- assets
- data
- kfto-scripts
- pipeline
- ray-scripts
- setup
- utils
- workshop
- 📄 0_sandbox.ipynb
- 📄 1_experiment_train.ipynb
- 📄 2_save_model.ipynb
- 📄 3_rest_requests.ipynb
- 🔗 4 Train Save.pipeline
- 📄 5_get_data_train_upload.yaml
- 📄 6_distributed_training.ipynb
- 📄 7_distributed_training_kfto.ipynb
- 📄 codeflare_sdk-0.34.0-py3-none-any.whl
- 📄 LICENSE
- 📄 README.md

2. Verify that the Git version at the bottom of the JupyterLab window is **v3.2**.

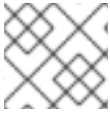
Next step

[Running code in a notebook](#)

or

[Training a model](#)

3.3. RUNNING CODE IN A NOTEBOOK

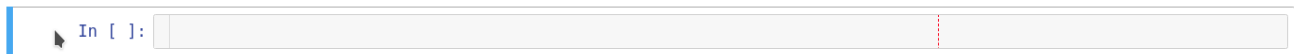


NOTE

If you're already at ease with Jupyter, you can [skip to the next section](#).

A notebook is an environment where you have *cells* that can display formatted text or code.

This is an empty cell:



This is a cell with some code:



Code cells contain Python code that you can run interactively. You can edit the code and then run it. The code does not run on your computer or in the browser, but directly in your connected environment, **Red Hat OpenShift AI** in your case.

You can run a code cell from the notebook interface or from the keyboard:

- **From the user interface:** Select the cell (by clicking inside the cell or to the left side of the cell) and then click **Run** from the toolbar.



- **From the keyboard:** Press **CTRL + ENTER** to run a cell or press **SHIFT + ENTER** to run the cell and automatically select the next one.

After you run a cell, you can see the result of its code and information about when the code in the cell ran, as shown in this example:



When you save a notebook, the code and the results are saved. You can reopen the notebook to view the results without having to run the program again, and still having access to the code.

Notebooks are so named because they are like a physical *notebook*: you can take notes about your

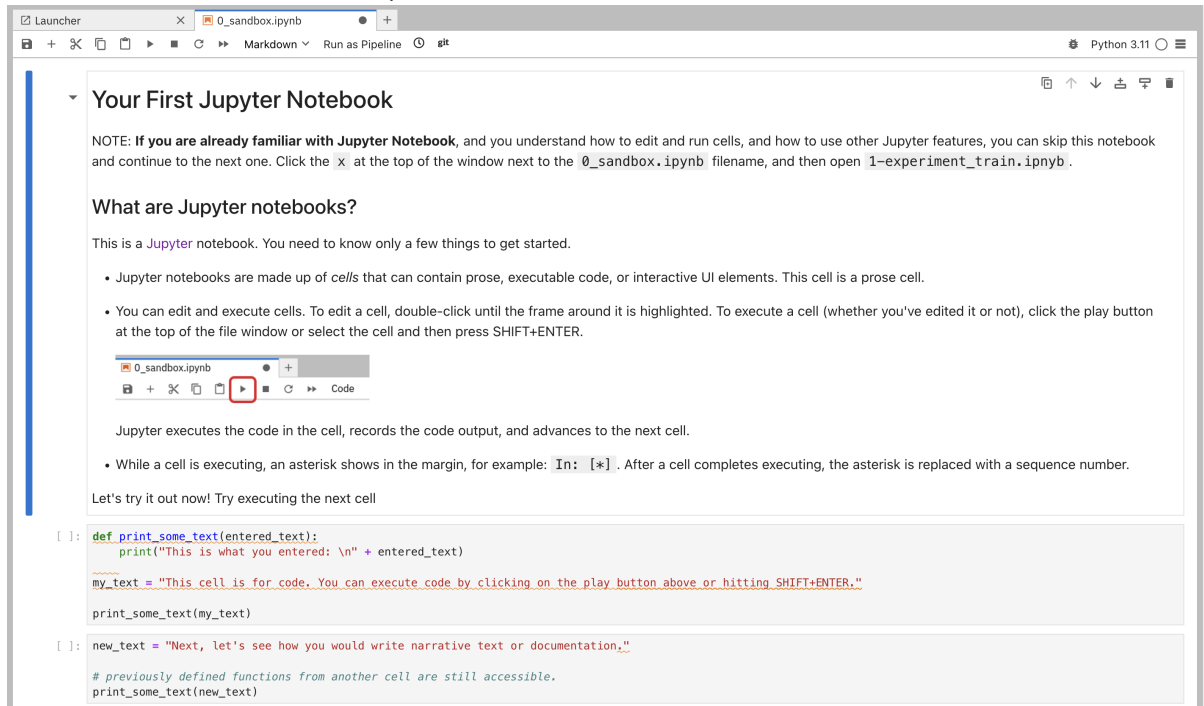
experiments, along with the code itself, including any parameters that you set. You can see the output of the experiment inline (this is the result after a cell runs), along with all the notes that you want to take. To take notes, from the menu switch the cell type from **Code** to **Markdown**.

Prerequisites

- You have imported the tutorial files into your JupyterLab environment as described in [Importing the tutorial files into the JupyterLab environment](#).

Procedure

- In your JupyterLab environment, locate the **0_sandbox.ipynb** file and double-click it to launch the notebook. The notebook opens in a new tab in the content section of the environment.



- Experiment by, for example, running the existing cells, adding more cells and creating functions. You can do what you want – it is your environment and there is no risk of breaking anything or impacting other users. This environment isolation is also a great advantage brought by OpenShift AI.
- Optionally, create a new notebook in which the code cells are run by using a Python 3 kernel:
 - Create a new notebook by either selecting **File → New → Notebook** or by clicking the Python 3 tile in the Notebook section of the launcher window:



You can use different kernels, with different languages or versions, to run in your notebook.

Additional resources

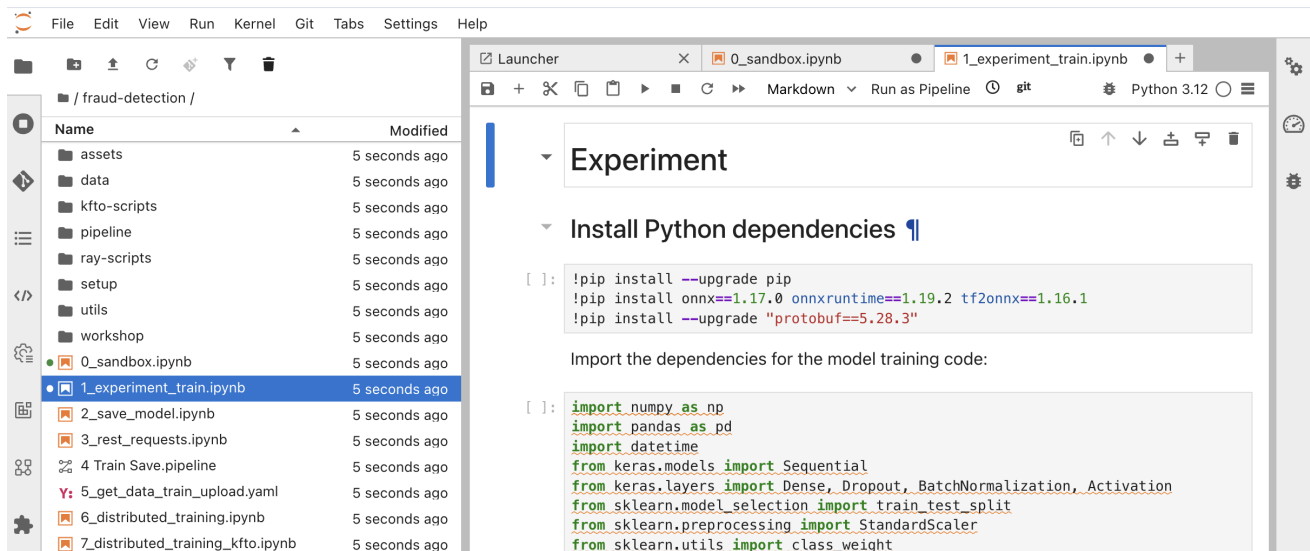
- [the Jupyter site](#)

Next step

[Training a model](#)

3.4. TRAINING A MODEL

In your notebook environment, open the **1_experiment_train.ipynb** file and follow the instructions directly in the notebook. The instructions guide you through some simple data exploration, experimentation, and model training tasks.



When you save the model, you convert the model to the portable Open Neural Network Exchange (ONNX) format. By using ONNX, you can transfer models between frameworks with minimal preparation and without the need for rewriting the models.

Next step

[Preparing the model for deployment](#)

CHAPTER 4. DEPLOYING AND TESTING A MODEL

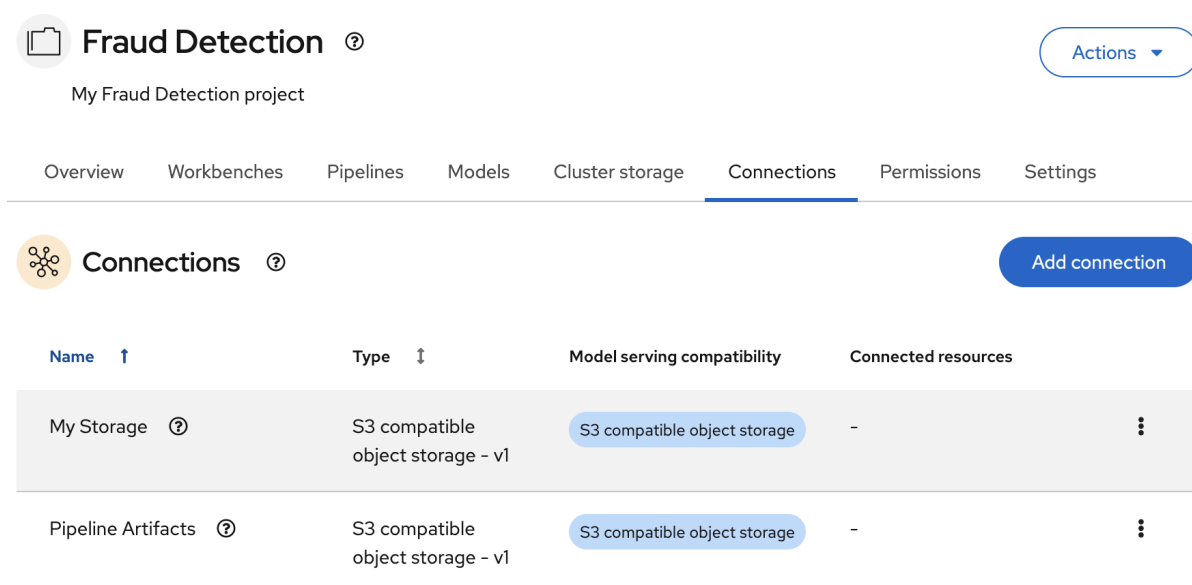
4.1. PREPARING THE MODEL FOR DEPLOYMENT

After you train a model, you can deploy it by using the OpenShift AI model serving capabilities. Model serving in OpenShift AI requires that you store models in object storage so that the model server pods can access them.

To prepare a model for deployment, you must move the model from your workbench to your S3-compatible object storage. Use the connection that you created in the [Storing data with connections](#) section and upload the model from a notebook.

Prerequisites

- You created the **My Storage** connection and have added it to your workbench.



The screenshot shows the OpenShift AI console interface. At the top, there's a header for the 'Fraud Detection' project with an 'Actions' button. Below this is a navigation bar with tabs: Overview, Workbenches, Pipelines, Models, Cluster storage, **Connections**, Permissions, and Settings. The 'Connections' tab is active, showing a table of connections. The table has columns: Name, Type, Model serving compatibility, and Connected resources. Two connections are listed: 'My Storage' and 'Pipeline Artifacts', both of type 'S3 compatible object storage - v1' and using 'S3 compatible object storage' for model serving compatibility. An 'Add connection' button is visible in the top right of the connections section.

Name	Type	Model serving compatibility	Connected resources
My Storage	S3 compatible object storage - v1	S3 compatible object storage	-
Pipeline Artifacts	S3 compatible object storage - v1	S3 compatible object storage	-

Procedure

- In your JupyterLab environment, open the **2_save_model.ipynb** file.
- Follow the instructions in the notebook to make the model accessible in storage.

Verification

When you have completed the notebook instructions, the **models/fraud/1/model.onnx** file is in your object storage and it is ready for your model server to use.

Next step

[Deploying the model](#)

4.2. DEPLOYING THE MODEL

You can use an OpenShift AI model server to deploy the model as an API.

Prerequisites

- You have saved the model as described in [Preparing the model for deployment](#).
- You have installed KServe and enabled the model serving platform.
- You have enabled a preinstalled or custom model-serving runtime.
- You have obtained values for the following MinIO storage parameters:
 - Access Key
 - Secret Key
 - Endpoint
 - Region
 - BucketTo obtain these values, navigate to your project's **Connections** tab. For the MyStorage connection, click the action menu (⋮) and then click **Edit**.

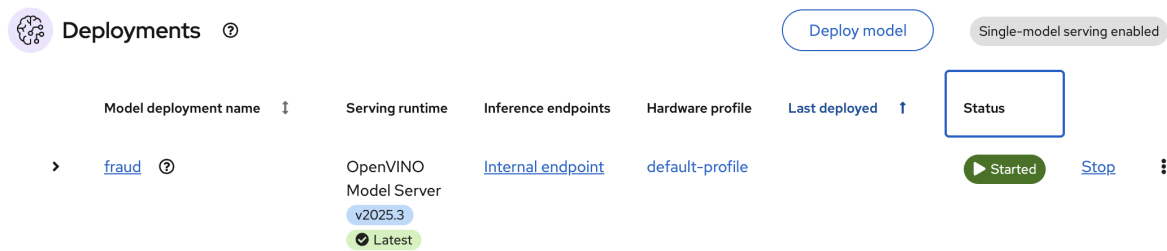
Procedure

1. In the OpenShift AI dashboard, navigate to the project details page and click the **Deployments** tab.
2. Click **Deploy model**.
The **Deploy a model** wizard opens.
3. In the **Model details** section, provide information about the model:
 - a. For **Model location**, select **Existing connection** and then select **My Storage**.
 - b. Enter the following values from your MinIO storage connection:
 - Access Key
 - Secret Key
 - Endpoint
 - Region
 - Bucket
 - c. For **Path**, enter **models/fraud**.
 - d. For **Model type**, select **Predictive model**.
 - e. Click **Next**.
4. In the **Model deployment** section, configure the deployment:
 - a. For **Model deployment name**, enter **fraud**.
 - b. For **Description**, enter a description of your deployment.
 - c. For the hardware profile, keep the default value.

- d. For **Model framework (name - version)**, select **onnx-1**.
 - e. For the **Serving runtime** field, accept the auto-selected runtime, **OpenVINO Model Server**.
 - f. Click **Next**.
5. In the **Advanced settings** section, accept the defaults by clicking **Next**.
 6. In the **Review** section, click **Deploy model**.

Verification

- Confirm that the deployed model is shown on the **Deployments** tab for the project, and on the **Deployments** page of the dashboard with a **Started** status.



Next step

[Testing the model API](#)

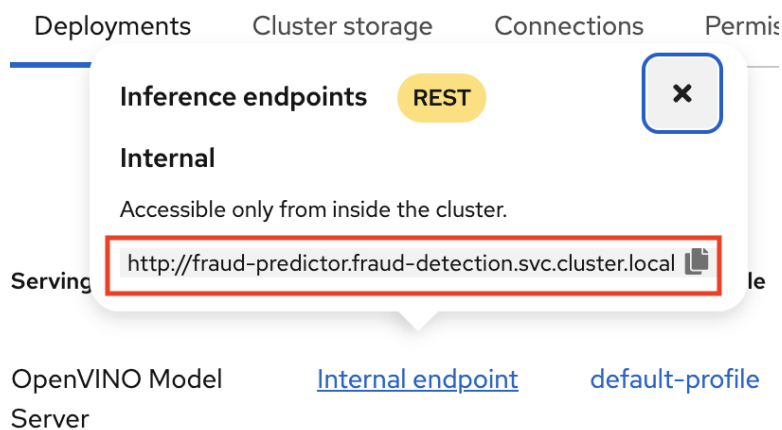
4.3. TESTING THE MODEL API

After you deploy the model, you can test its API endpoints.

Procedure

- In the OpenShift AI dashboard, navigate to the project details page and click the **Deployments** tab.
- Take note of the model's Inference endpoint URL. You need this information when you test the model API.

If the **Inference endpoint** field has an **Internal endpoint details** link, click the link to open a text box that shows the URL details, and then take note of the **restUrl** value.



NOTE: When you test the model API from inside a workbench, you must edit the endpoint to specify **8888** for the port. For example:

`http://fraud-predictor.fraud-detection.svc.cluster.local:8888`

3. Return to the JupyterLab environment and try out your new endpoint.
Follow the directions in **`3_rest_requests.ipynb`** to try a REST API call.

Next step

(Optional) [Automating workflows with AI pipelines](#)

(Optional) [Running a pipeline generated from Python code](#)

CHAPTER 5. IMPLEMENTING PIPELINES

5.1. AUTOMATING WORKFLOWS WITH AI PIPELINES

Earlier, you used a notebook to train and save your model. Optionally, you can automate these tasks by using Red Hat OpenShift AI pipelines. Pipelines automate the execution of multiple notebooks and Python code. By using pipelines, you can run long training jobs or retrain your models on a schedule without having to manually run them in a notebook.

To explore the pipeline editor, complete the steps in the following procedures to create your own pipeline.

Alternately, you can skip the following procedures and instead run the **4 Train Save.pipeline** file.

5.1.1. Creating a pipeline

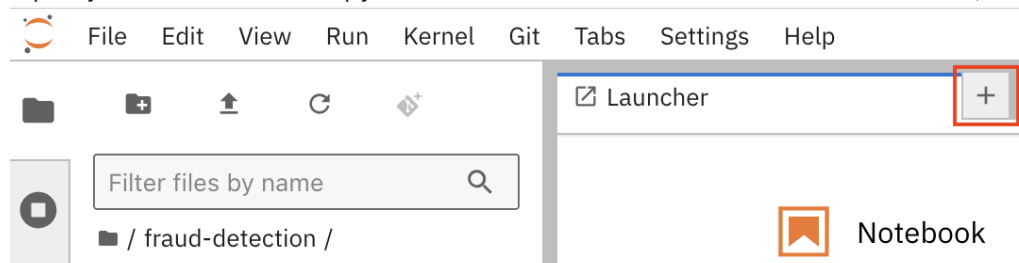
You can create a pipeline by using the GUI pipeline editor. This pipeline automates the notebook workflow that you used earlier to train a model and save it to S3 storage.

Prerequisites

- You configured a pipeline server as described in [Enabling AI pipelines](#).
- If you configured the pipeline server after you created your workbench, you stopped and then started your workbench.

Procedure

1. Open your workbench's JupyterLab environment. If the launcher is not visible, click + to open it.

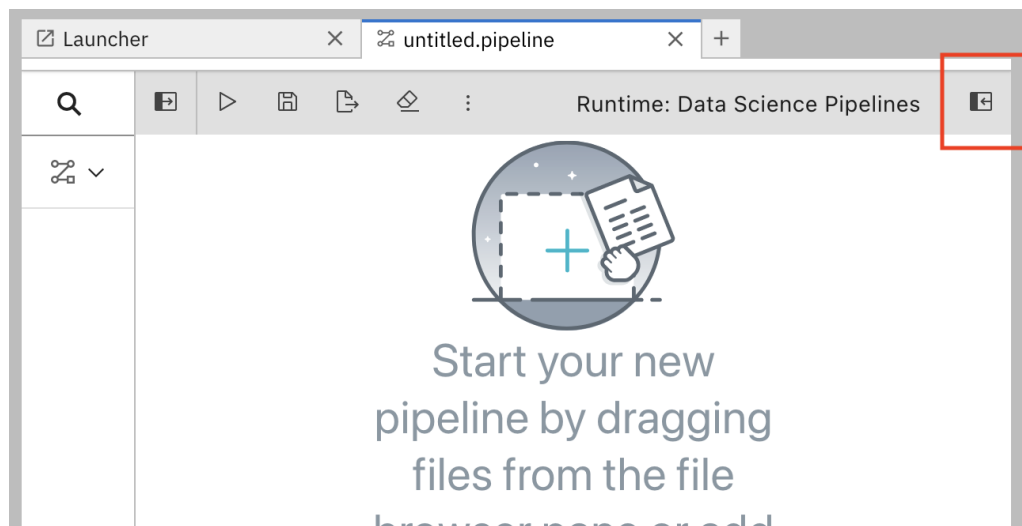


2. Click **Pipeline Editor**.

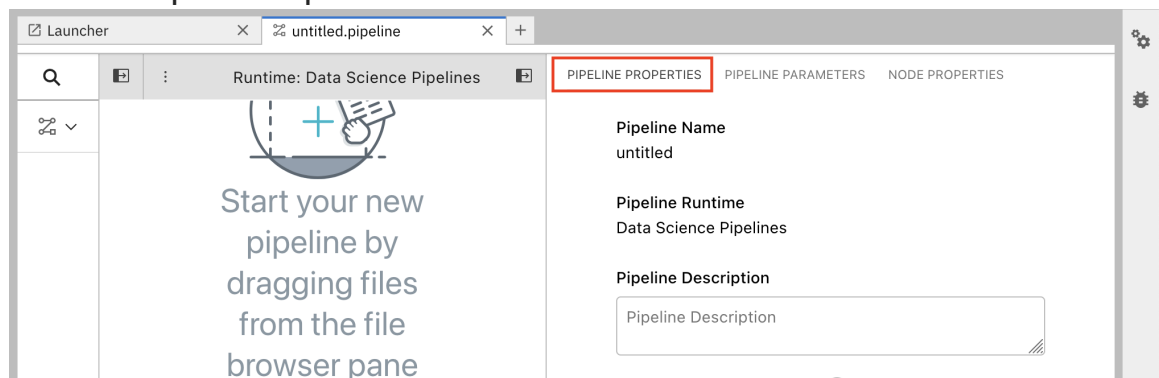


You have created a blank pipeline.

3. Set the default runtime image for when you run your notebook or Python code.
 - a. In the pipeline editor, click **Open Panel**.



b. Select the **Pipeline Properties** tab.



c. In the **Pipeline Properties** panel, scroll down to **Generic Node Defaults** and **Runtime Image**. Set the value to **Runtime | Tensorflow | Cuda | Python 3.12**.

Generic Node Defaults ?

Runtime Image ?

Runtime | TensorFlow | CUDA | Python 3.12 ▼

4. Select **File** → **Save Pipeline**.

Verification

- In the JupyterLab file browser, the pipeline file (for example, **untitled.pipeline**) appears in your working directory.

Next step

[Adding nodes to your pipeline](#)

5.1.2. Adding nodes to your pipeline

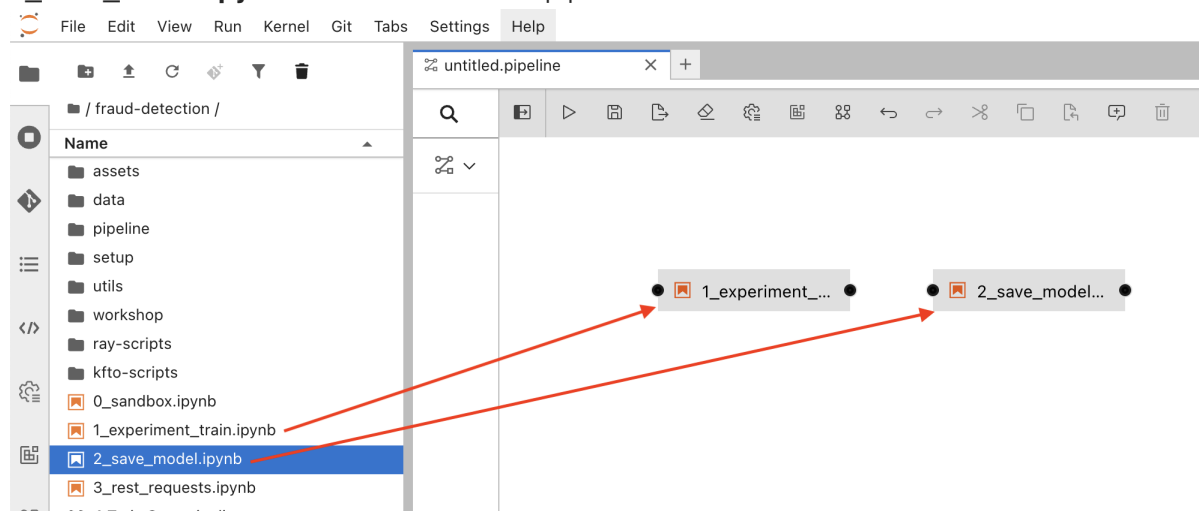
Add some steps, or **nodes**, in your pipeline for the **1_experiment_train.ipynb** and **2_save_model.ipynb** notebooks.

Prerequisites

- You created a pipeline file as described in [title](#)

Procedure

- From the JupyterLab file-browser panel, drag the **1_experiment_train.ipynb** and **2_save_model.ipynb** notebooks onto the pipeline canvas.



- Click the output port of **1_experiment_train.ipynb** and drag a connecting line to the input port of **2_save_model.ipynb**.



- Save the pipeline.

Verification

- Your pipeline has two nodes.

Next step

[Specify the training file as a dependency](#)

5.1.3. Specifying the training file as a dependency

Set node properties in your pipeline to specify the training file as a dependency.



NOTE

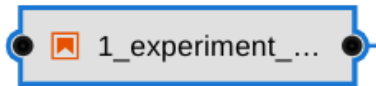
If you do not set this file dependency, the file is not included in the node when it runs and the training job fails.

Prerequisites

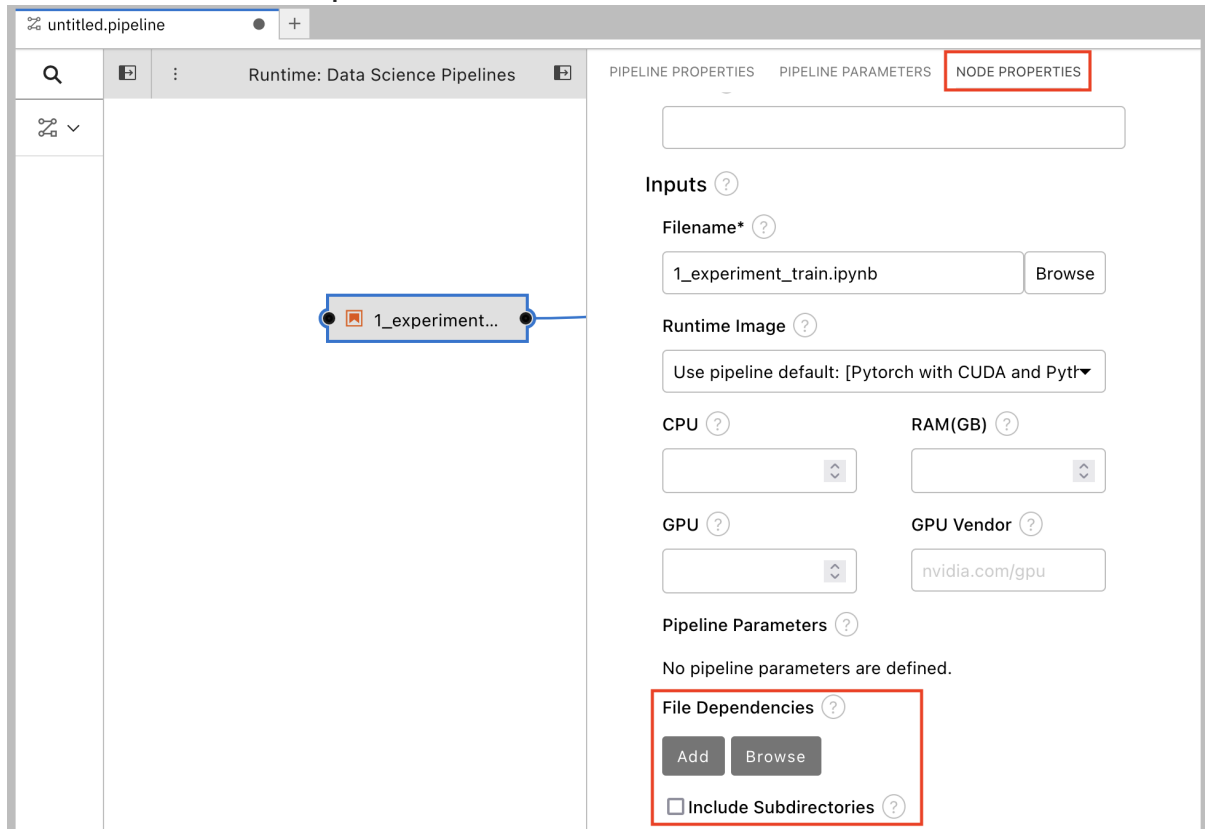
- You added two nodes to your pipeline as described in [Adding nodes to your pipeline](#).

Procedure

1. Click the **1_experiment_train.ipynb** node.



2. In the **Properties** panel, click the **Node Properties** tab.
3. Scroll down to the **File Dependencies** section and then click **Add**.



4. Set the value to **data/*.csv** which has the data to train your model.
5. Select the **Include Subdirectories** option.

File Dependencies ?

data/*.csv

Add Browse

☒ Include Subdirectories ?

6. **Save** the pipeline.

Verification

- The training file is a dependency of the first node in your pipeline.

Next step

[Creating and storing the ONNX-formatted output file](#)

5.1.4. Creating and storing the ONNX-formatted output file

You must set the **models/fraud/1/model.onnx** file as the output file for both nodes in your pipeline.

Prerequisites

- You added the training file as a dependency to the first node in your pipeline, as described in [Specifying the training file as a dependency](#).

Procedure

- Select node 1.
- Select the **Node Properties** tab.
- Scroll down to the **Output Files** section, and then click **Add**.
- Set the value to **models/fraud/1/model.onnx**.

Outputs ?

Output Files ?

models/fraud/1/model.onnx

Remove

Add

- Repeat steps 2-4 for node 2.
- Click **Save Pipeline**.

Verification

- In node 1, the notebook creates the **models/fraud/1/model.onnx** file.
- In node 2, the notebook uploads the **models/fraud/1/model.onnx** file to the S3 storage bucket.

Next step

[Configuring the connection to storage](#)

5.1.5. Configuring the connection to storage

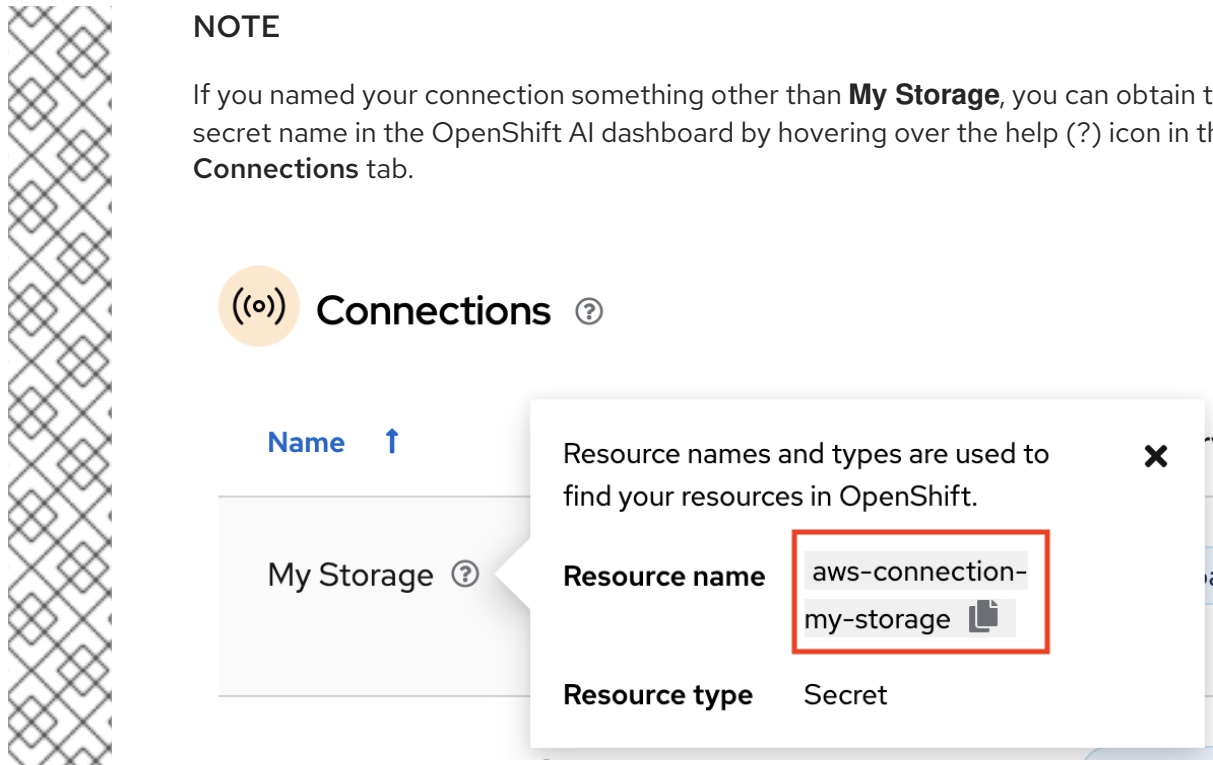
In node 2, the notebook uploads the model to the S3 storage bucket. You must set the S3 storage bucket keys by using the secret created by the **My Storage** connection that you set up in [Storing data with connections](#).

You can use this secret in your pipeline nodes without having to save the information in your pipeline code. This is important, for example, if you want to save your pipelines – without any secret keys – to source control.

The name of the secret is **my-storage**.

NOTE

If you named your connection something other than **My Storage**, you can obtain the secret name in the OpenShift AI dashboard by hovering over the help (?) icon in the **Connections** tab.



The **my-storage** secret includes the following fields:

- **AWS_ACCESS_KEY_ID**
- **AWS_DEFAULT_REGION**
- **AWS_S3_BUCKET**
- **AWS_S3_ENDPOINT**
- **AWS_SECRET_ACCESS_KEY**

You must set the secret name and key for each of these fields.

Prerequisites

- You created the **My Storage** connection, as described in [Storing data with connections](#).
- You set the **models/fraud/1/model.onnx** file as the output file for both nodes in your pipeline, as described in [Creating and storing the ONNX-formatted output file](#).

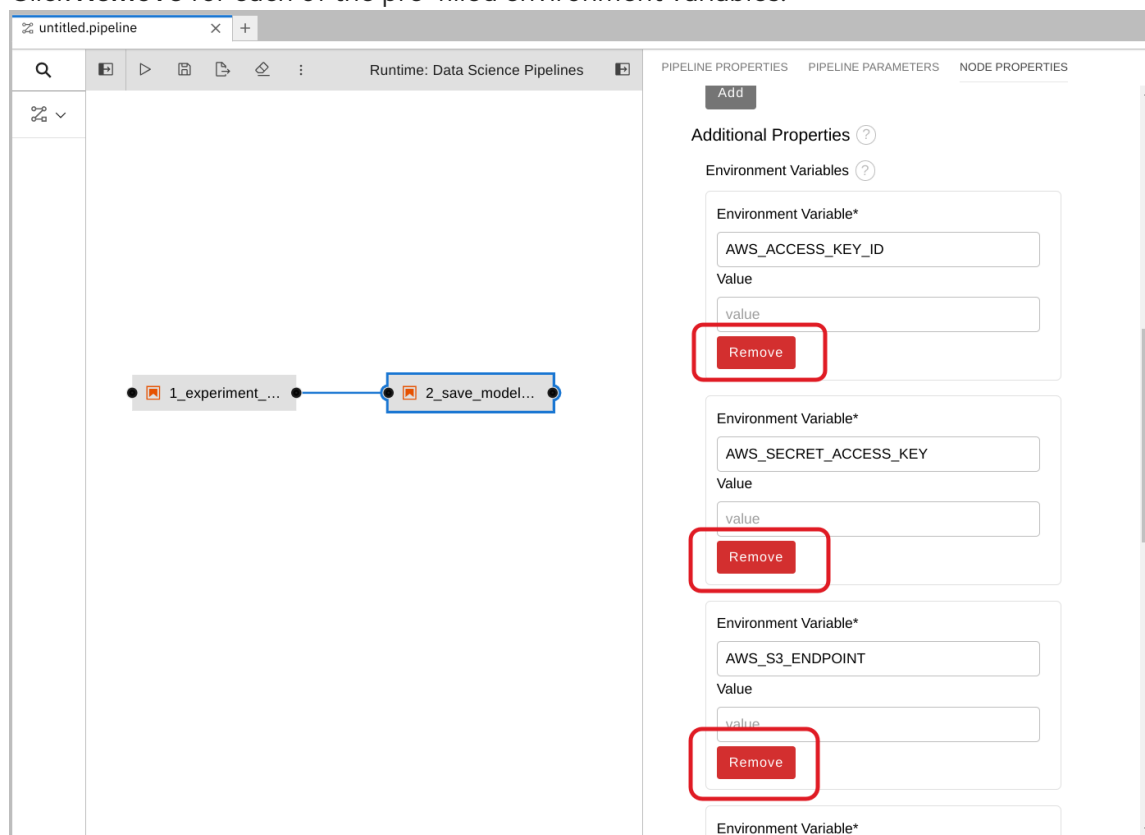
Procedure

1. Remove any pre-filled environment variables.
 - a. Select node 2, and then select the **Node Properties** tab.

Under **Additional Properties**, note that some environment variables have been pre-filled. The pipeline editor inferred that you need them from the notebook code.

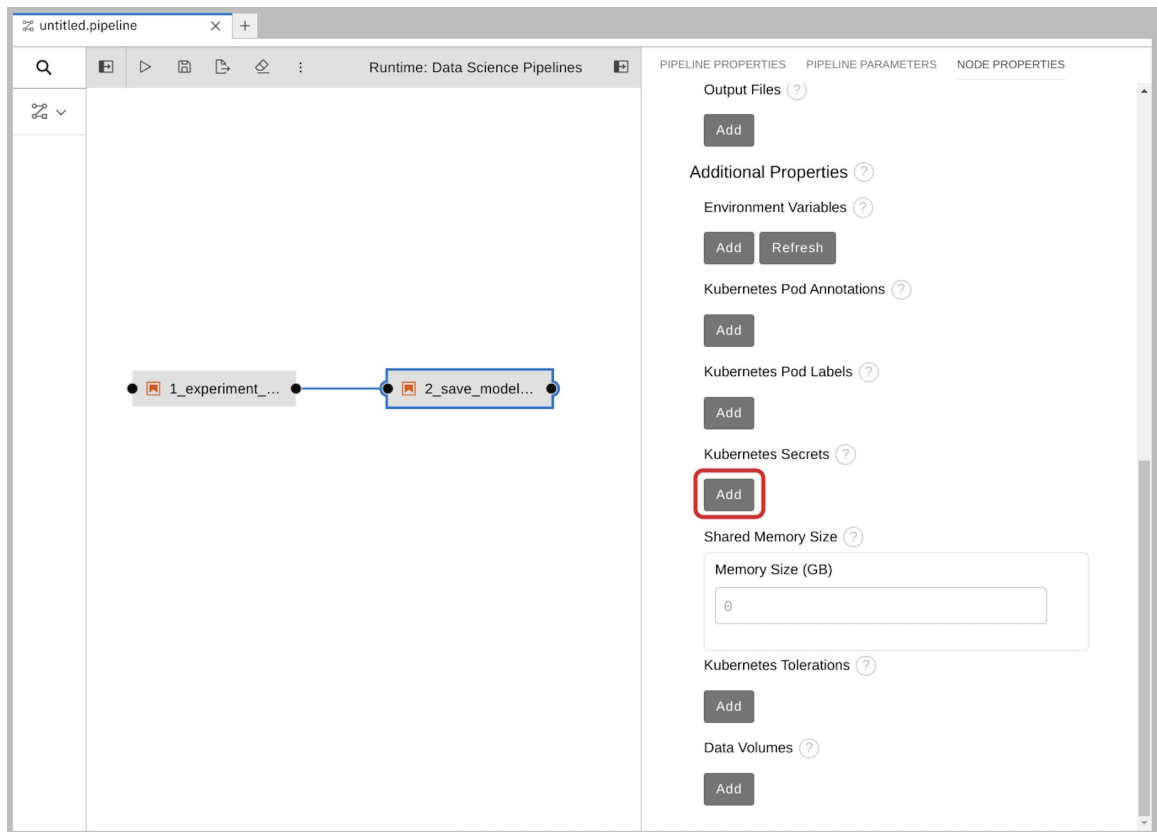
Because you do not want to save the value in your pipelines, remove all of these environment variables.

- b. Click **Remove** for each of the pre-filled environment variables.



2. Add the S3 bucket and keys by using the Kubernetes secret.

- a. Under **Kubernetes Secrets**, click **Add**.



b. Enter the following values and then click **Add**.

- Environment Variable: **AWS_ACCESS_KEY_ID**
 - Secret Name: **my-storage**
 - Secret Key: **AWS_ACCESS_KEY_ID**

Kubernetes Secrets ?

Environment Variable*

Secret Name*

Secret Key*

Remove

Add

3. Repeat Step 2 for each of the following Kubernetes secrets:

- Environment Variable: **AWS_SECRET_ACCESS_KEY**
 - Secret Name: **my-storage**
 - Secret Key: **AWS_SECRET_ACCESS_KEY**
- Environment Variable: **AWS_S3_ENDPOINT**
 - Secret Name: **my-storage**
 - Secret Key: **AWS_S3_ENDPOINT**
- Environment Variable: **AWS_DEFAULT_REGION**
 - Secret Name: **my-storage**
 - Secret Key: **AWS_DEFAULT_REGION**
- Environment Variable: **AWS_S3_BUCKET**
 - Secret Name: **my-storage**
 - Secret Key: **AWS_S3_BUCKET**

4. Select **File** → **Save Pipeline As** to save and rename the pipeline. For example, rename it to **My Train Save.pipeline**.

Verification

- In the **Node Properties** tab for node 2, the **Kubernetes Secrets** section lists all five environment variables configured with the **my-storage** secret.

Next step

[Running your pipeline](#)

5.1.6. Running your pipeline

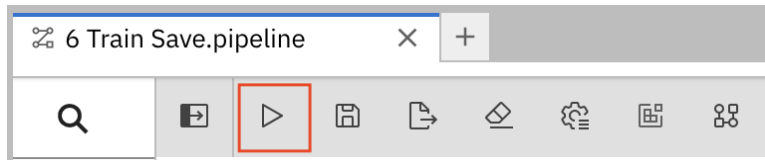
You can upload and run your pipeline directly from the pipeline editor. Use either your newly created pipeline or the provided **4 Train Save.pipeline** file.

Prerequisites

- You set the S3 storage bucket keys, as described in [Configuring the connection to storage](#) .
- You configured a pipeline server, as described in [Enabling AI pipelines](#).
- If you created your workbench before the pipeline server was available, you stopped and restarted the workbench.

Procedure

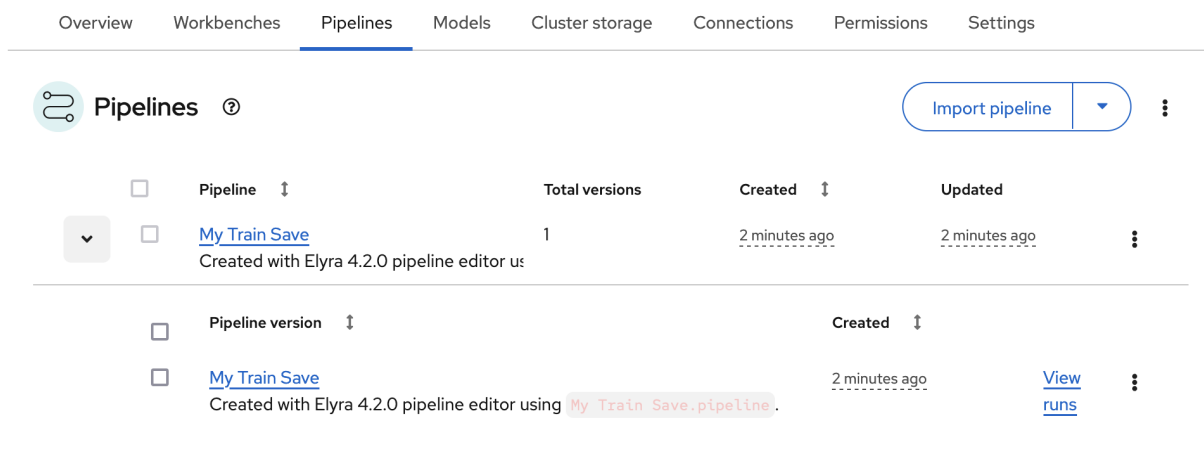
1. In the JupyterLab pipeline editor toolbar, click **Run Pipeline**.



2. Enter a name for your pipeline.
3. Verify that the **Runtime Configuration:** is set to **Pipeline**.
4. Click **OK**.

NOTE: If you see an error message stating that "no runtime configuration for Pipelines is defined", you might have created your workbench before the pipeline server was available. To address this error, you must verify that you configured the pipeline server and then restart the workbench.

5. In the OpenShift AI dashboard, open your project and expand the newly created pipeline.



6. Click **View runs**.

Manage your experiment runs and schedules.

Project Fraud Detection ▼

Runs Archive Schedules

Run ▼

Search...

Create run

Pipeline version My Train Save ✕

[Clear all filters](#)

☐	Run ↕	Pipeline version	Experin
☐	My Train Save-0120013824 One-off	My Train Save	my trai

7. Click your run and then view the pipeline run in progress.

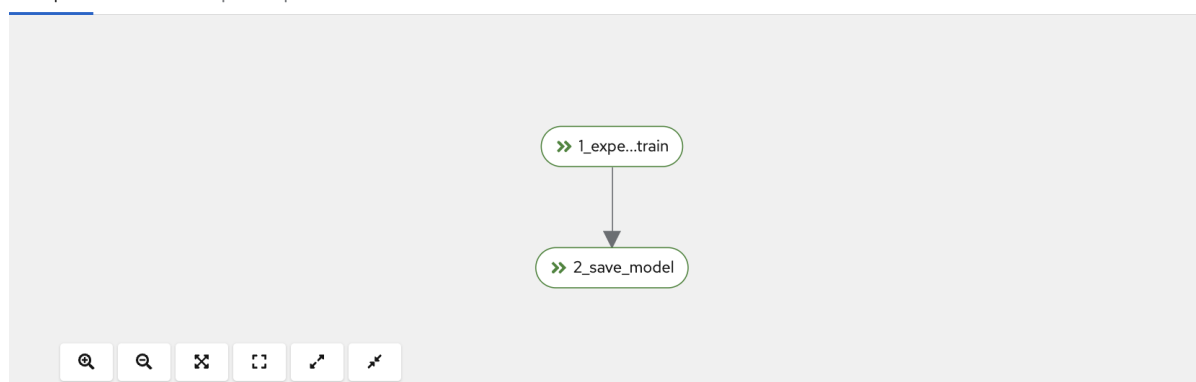
MyTrainSave1-1024180103

One-off

Running

Actions ▼

Graph Details Pipeline spec



Verification

- The **models/fraud/1/model.onnx** file is in your S3 bucket.

Next step

- (Optional) [Running a pipeline generated from Python code](#)

- Serve the model, as described in [Preparing a model for deployment](#).

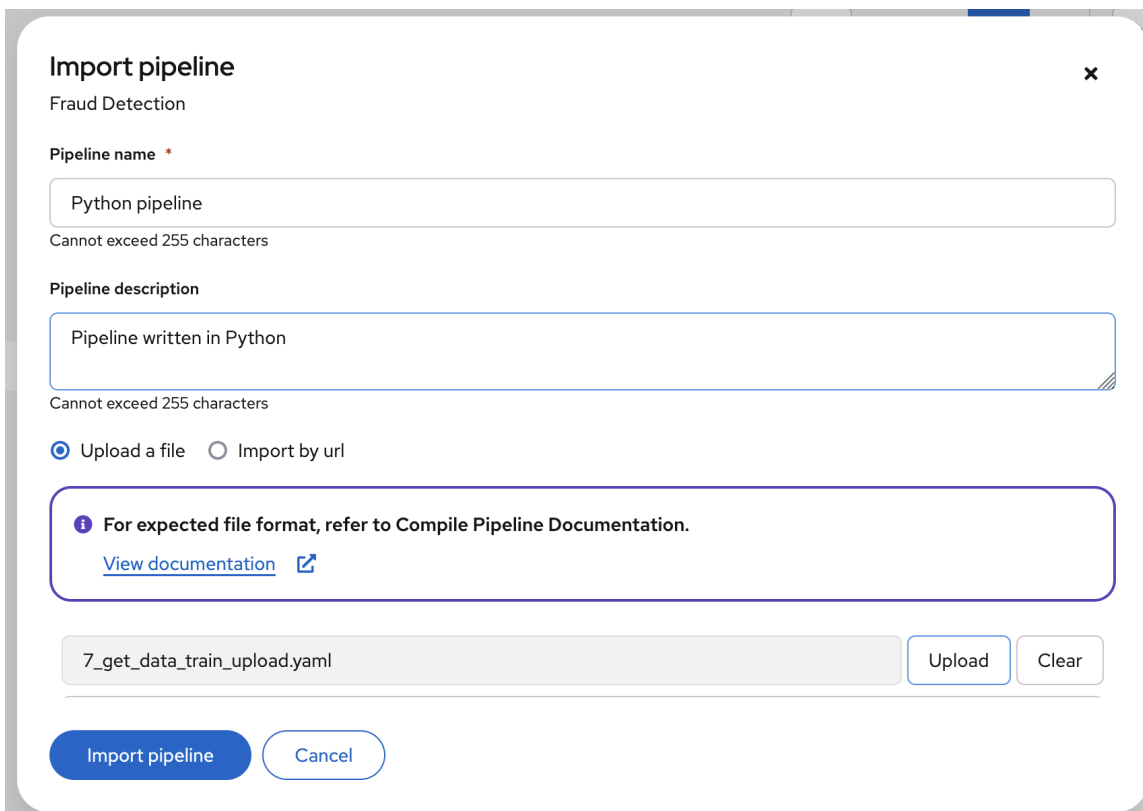
5.2. RUNNING A PIPELINE GENERATED FROM PYTHON CODE

Earlier, you created a simple pipeline by using the GUI pipeline editor. You might want to create pipelines by using code that can be version-controlled and shared with others. The [Kubeflow pipelines \(kfp\)](#) SDK provides a Python API for creating pipelines. The SDK is available as a Python package. With this package, you can use Python code to create a pipeline and then compile it to YAML format. Then you can import the YAML code into OpenShift AI.

This tutorial does not describe the details of how to use the SDK. Instead, it provides the files for you to view and upload.

Procedure

1. Optionally, view the provided Python code in your JupyterLab environment by navigating to the **fraud-detection-notebooks** project's **pipeline** directory. It contains the following files:
 - **5_get_data_train_upload.py** is the main pipeline code.
 - **build.sh** is a script that builds the pipeline and creates the YAML file.
For your convenience, the output of the **build.sh** script is provided in the **5_get_data_train_upload.yaml** file. The **5_get_data_train_upload.yaml** output file is located in the top-level **fraud-detection** directory.
2. Right-click the **5_get_data_train_upload.yaml** file and then click **Download**.
3. Upload the **5_get_data_train_upload.yaml** file to OpenShift AI.
 - a. In the OpenShift AI dashboard, navigate to your project page. Click the **Pipelines** tab and then click **Import pipeline**.
 - b. Enter values for **Pipeline name** and **Pipeline description**.
 - c. Click **Upload** and then select **5_get_data_train_upload.yaml** from your local files to upload the pipeline.



Import pipeline ✕

Fraud Detection

Pipeline name *

Python pipeline

Cannot exceed 255 characters

Pipeline description

Pipeline written in Python

Cannot exceed 255 characters

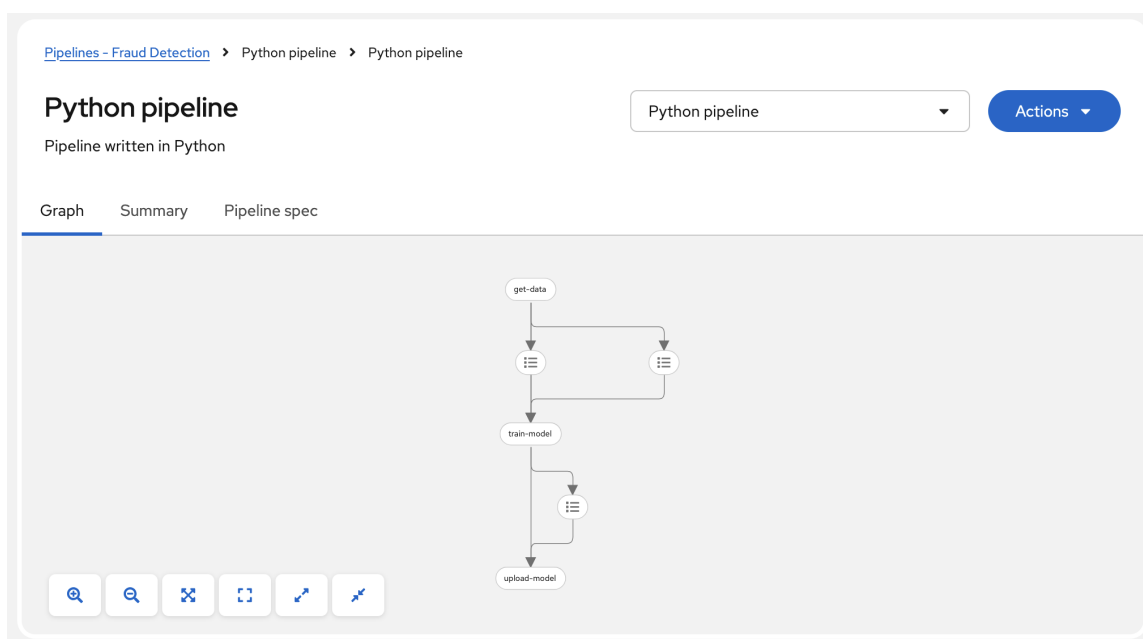
☒ Upload a file ☐ Import by url

i For expected file format, refer to [Compile Pipeline Documentation](#).
[View documentation](#) [🔗](#)

7_get_data_train_upload.yaml Upload Clear

Import pipeline Cancel

- d. Click **Import pipeline** to import and save the pipeline.
 The pipeline shows in graphic view.



4. Select **Actions** → **Create run**.
5. On the **Create run** page, provide the following values:
 - a. For **Experiment**, leave the value as **Default**.
 - b. For **Name**, type any name, for example **Run 1**.
 - c. For **Pipeline**, select the pipeline that you uploaded.
 You can leave the other fields with their default values.

Create run

Jump to section

Run type

Project and experiment

Run details

Pipeline

Parameters

Run type

Run once immediately after creation

Project and experiment

Project

Fraud Detection

Experiment *

Default

[+ Create new experiment](#)

Run details

Name *

Run 1

Cannot exceed 255 characters

Description

Cannot exceed 255 characters

Pipeline

Pipeline *

Python pipeline

Create run

Cancel

- Click **Create run** to create the run.

Verification

- A new run starts immediately.

[Runs - Fraud Detection](#) > Run 1

Run 1

One-off

 Running

Graph

Details

Input parameters

Pipeline spec

CHAPTER 6. RUNNING A DISTRIBUTED WORKLOAD

You can distribute the training of a machine learning model across many CPUs by using Ray or the Training Operator.

6.1. DISTRIBUTING TRAINING JOBS

Earlier, you trained the fraud detection model directly in a notebook and then in a pipeline. You can also distribute the training of a machine learning model across many CPUs.

Distributing training is not necessary for a simple model. However, by applying it to the example fraud model, you learn how to train more complex models that require more compute power.

NOTE: Distributed training in OpenShift AI uses the Red Hat build of Kueue for admission and scheduling. Before you run the Ray or Training Operator examples in this tutorial, complete the setup tasks in [Setting up Kueue resources](#).

You can try one or both of the following options:

- The Ray distributed computing framework, as described in [Distributing training jobs with Ray](#).
- The Training Operator, as described in [Distributing training jobs with the Training Operator](#).

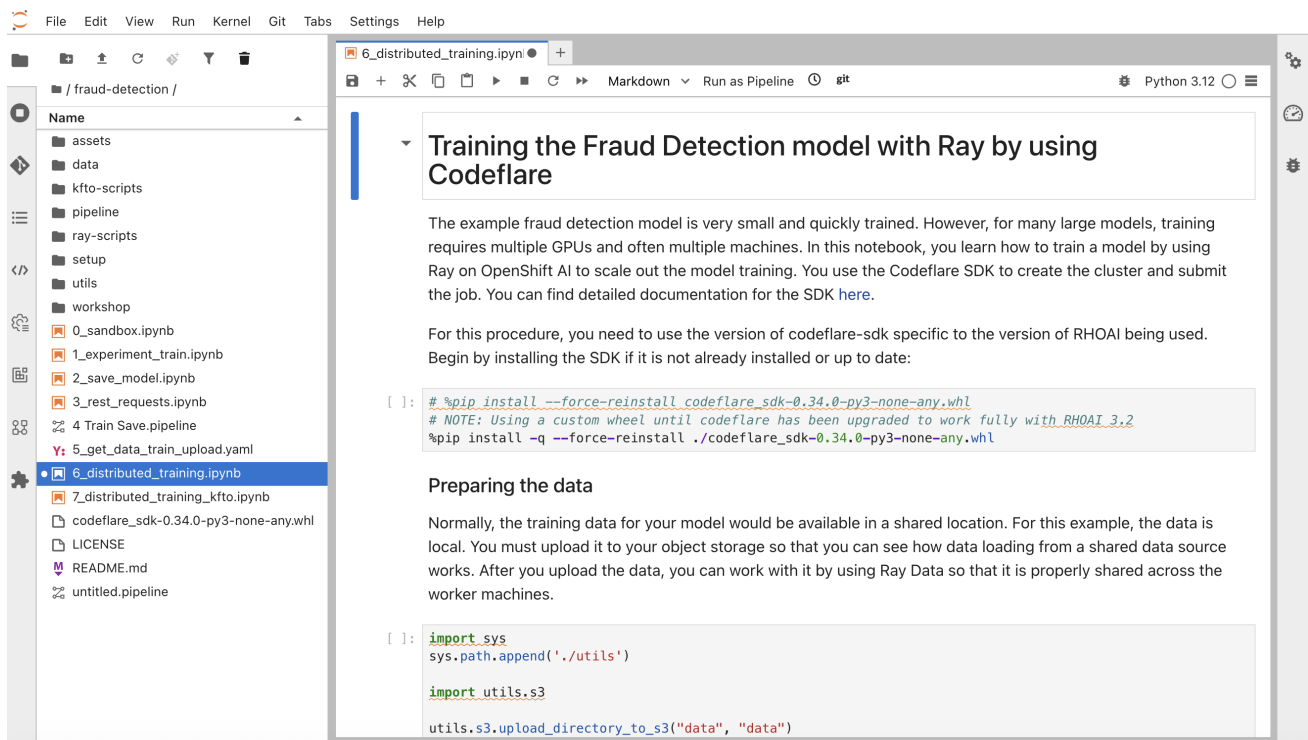
6.1.1. Distributing training jobs with Ray

You can use Ray, a distributed computing framework, to parallelize Python code across many CPUs or GPUs.

NOTE: Distributed training in OpenShift AI uses the Red Hat build of Kueue for admission and scheduling. Before you run a distributed training example in this tutorial, complete the setup tasks in [Setting up Kueue resources](#).

In your notebook environment, open the **6_distributed_training.ipynb** file and follow the instructions directly in the notebook. The instructions guide you through setting authentication, creating Ray clusters, and working with jobs.

Optionally, if you want to view the Python code for this step, you can find it in the **ray-scripts/train_tf_cpu.py** file.



For more information about TensorFlow training on Ray, see the [Ray TensorFlow guide](#).

6.1.2. Distributing training jobs with the Training Operator

The Training Operator is a tool for scalable distributed training of machine learning (ML) models created with various ML frameworks, such as PyTorch.

NOTE: Distributed training in OpenShift AI uses the Red Hat build of Kueue for admission and scheduling. Before you run a distributed training example in this tutorial, complete the setup tasks in [Setting up Kueue resources](#).

In your notebook environment, open the **7_distributed_training_kfto.ipynb** file and follow the instructions directly in the notebook. The instructions guide you through setting authentication, initializing the Training Operator client, and submitting a **PyTorchJob**.

You can also view the complete Python code in the **kfto-scripts/train_pytorch_cpu.py** file.

The screenshot shows a JupyterLab environment with a file browser on the left and a notebook editor on the right. The file browser shows a directory structure for 'fraud-detection' with various scripts and pipelines. The notebook editor displays the following content:

Training the Fraud Detection model with the Kubeflow Training Operator

The example fraud detection model is small and quickly trained. For many large models, training requires multiple GPUs and often multiple machines. In this notebook, you learn how to train a model by using the Kubeflow Training Operator on OpenShift AI to scale out model training. You use the Training Operator SDK to create a PyTorchJob that executes the provided model training script.

Install the Training Operator SDK

The Training Operator SDK is not available by default with the Tensorflow workbench image. Run the following command to install it:

```
[ ]: %pip install "kubeflow @ git+https://github.com/opendatahub-io/kubeflow-sdk.git@v0.2.1+rhai0"
```

```
[ ]: from kubeflow.trainer import TrainerClient
    from kubeflow.trainer.rhai import TransformersTrainer
```

```
[ ]: import sys
    import os
    sys.path.append("./kfto-scripts") # needed to make training function available in the notebook
    from train_pytorch_cpu import train_func
```

```
[ ]: from kubernetes import client
    from kubeflow.common.types import KubernetesBackendConfig

    api_server = "https://XXXX"
    token = "sha256~XXXX"
```

For more information about PyTorchJob training with the Training Operator, see the [Training Operator PyTorchJob guide](#).

CHAPTER 7. CONCLUSION

Congratulations. In this tutorial, you learned how to incorporate data science, artificial intelligence, and machine learning into an OpenShift development workflow.

You used an example fraud detection model and completed the following tasks:

- Explored a pre-trained fraud detection model by using a Jupyter notebook.
- Deployed the model by using OpenShift AI model serving.
- Refined and trained the model by using automated pipelines.
- Distributed model training across multiple nodes by using Ray and the Training Operator.